Anime Database

By: Jayden Dalglish, Ben DeZutti, Evan Marino

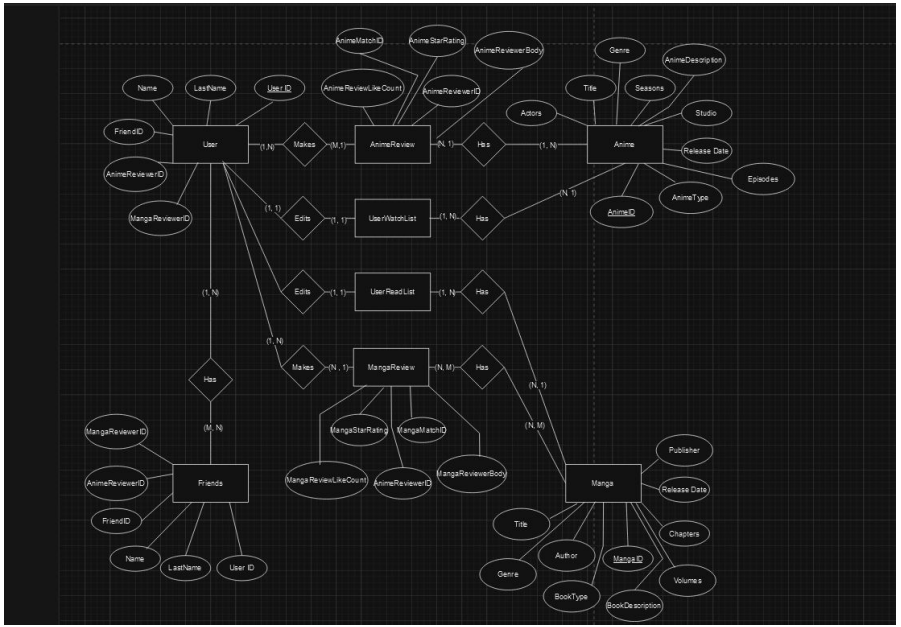
What’s AniWatch handling?

- Aniwatch is a database compiled of both Anime & Manga merging letterboxd and Goodreads for the types of media into one
- Anime doesn’t have a very good review website like books and movies do; this fixes that.

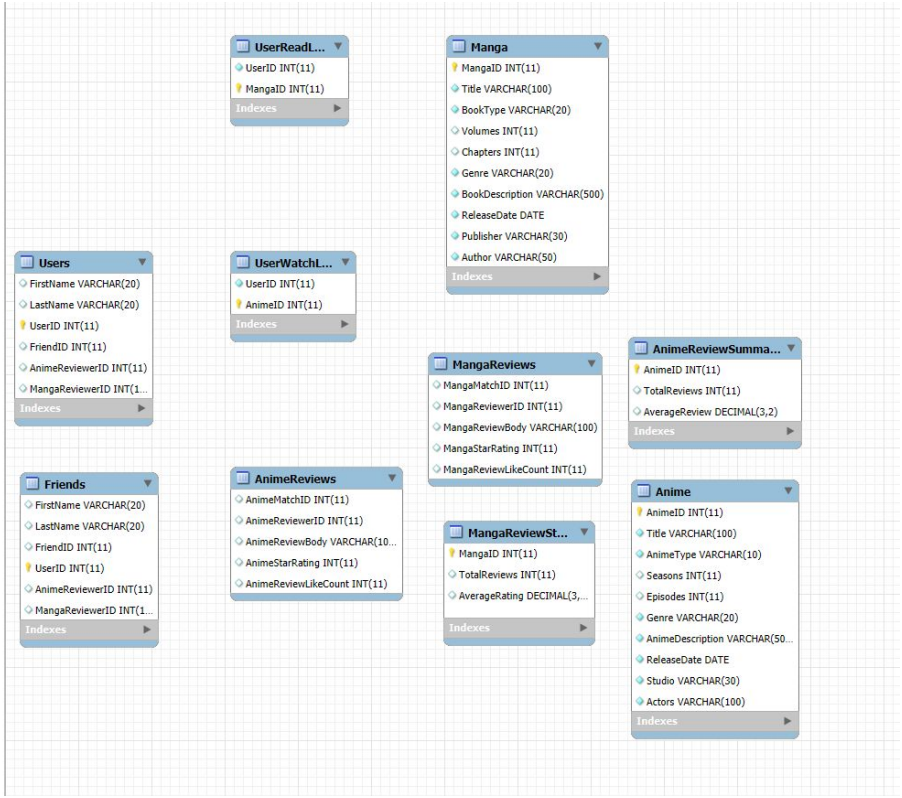
goodreads

Letterboxd

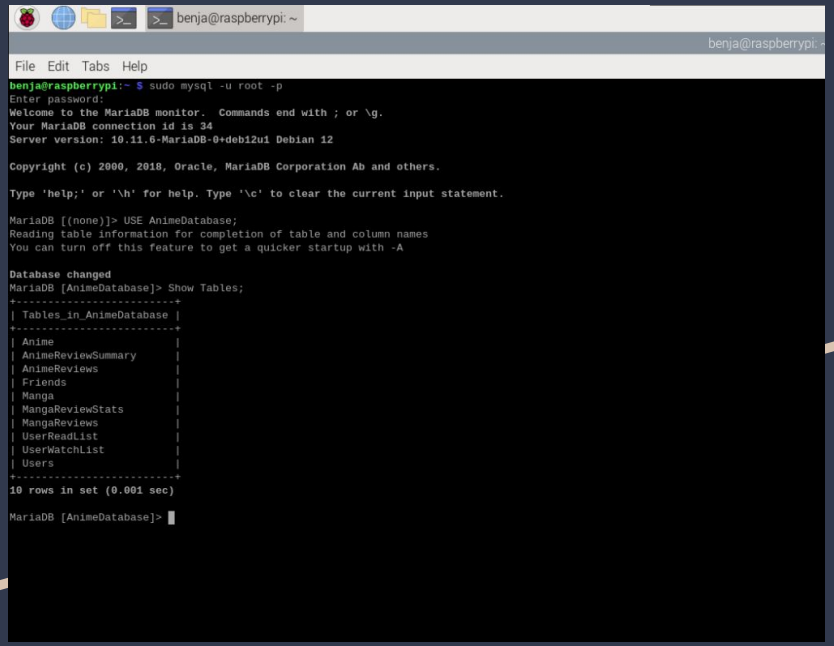
ER Diagram


AnimeMatchID
AnimeStarRating
AnimeReviewerBody
Genre
AnimeDescription
Name
LastName
User ID
AnimeReviewLikeCount
AnimeReviewerID
Title
Seasons
Actors
Studio
FriendID
User
(1, N)
Makes
(M,1)
AnimeReview
(N, 1)
Has
(1, N)
Anime
Release Date
AnimeReviewerID
Episodes
(N, 1)
AnimeType
MangaReviewerID
(1, 1)
Edits
(1, 1)
UserWatchList
(1, N)
Has
AnimeID
(1, N)
Edits
(1, 1)
UserReadList
(1, N)
Has
(1, N)
Makes
(N , 1)
MangaReview
(N, M)
Has
(N, 1)
Has
(M, N)
MangaStarRating
MangaMatchID
(N, M)
MangaReviewerID
Publisher
AnimeReviewerID
Friends
MangaReviewLikeCount
AnimeReviewerID
MangaReviewerBody
Manga
Release Date
FriendID
Title
Chapters
Author
MangaID
Name
LastName
User ID
Genre
Volumes
BookType
BookDescription

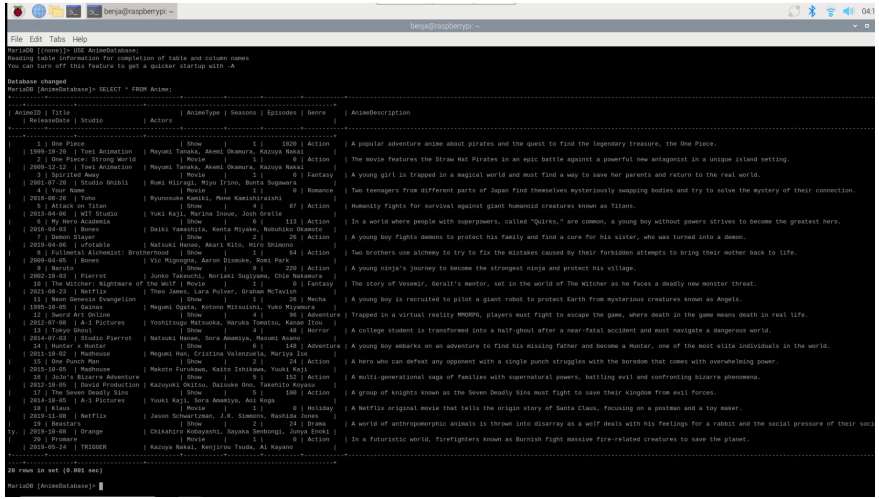
Physical Schema Diagram



MySQL Server Creation



- Made with a Raspberry Pi 4 and MariaDB
    - MariaDB is a stand-in for MySQL server, and is compatible with Debian (Raspberry Pi OS is based off of Debian)
- Made user accounts for teammates and provided them with all permissions
- Can do everything MySQL from command line



Anime and Manga Table Creation

- Primary Key for AnimeID as it's used later on for review and watch list
- Allows Description (with a character limit), including release date, studio, and voice actors that are in it

```sql
DROP TABLE IF EXISTS Anime;
CREATE TABLE Anime
(
AnimeID int NOT NULL,
Title varchar(100) NOT NULL,
AnimeType varchar(10) NOT NULL, -- Movie or Show
Seasons int,
Episodes int,
Genre varchar(20) NOT NULL,
AnimeDescription varchar(500) NOT NULL,
ReleaseDate date NOT NULL,
Studio varchar(30) NOT NULL,
Actors varchar(100) NOT NULL,
PRIMARY KEY (AnimeID)
);
```

```sql
DROP TABLE IF EXISTS Manga;
CREATE TABLE Manga
(
MangaID int NOT NULL,
Title varchar(100) NOT NULL,
BookType varchar(20) NOT NULL, -- Manga or Light Novel
Volumes int,
Chapters int,
Genre varchar(20) NOT NULL,
BookDescription varchar(500) NOT NULL,
ReleaseDate date NOT NULL,
Publisher varchar(30) NOT NULL,
Author varchar(50) NOT NULL,
PRIMARY KEY (MangaID)
);
```

Review Tables Creation

```sql
DROP TABLE IF EXISTS AnimeReviews;
CREATE TABLE AnimeReviews
(
AnimeMatchID int,
AnimeReviewerID int,
AnimeReviewBody VARCHAR(100),
AnimeStarRating int,
AnimeReviewLikeCount int
);
```

- No primary key used because each instance can happen multiple times
  - Reviewer can review many animes: animes can get reviewed by many Reviewers.
  - Include a small review log (Characters may need to be increased in the future)

User Table Creation along with Friend and Watchlist

```sql
DROP TABLE IF EXISTS UserWatchList;
CREATE TABLE UserWatchList
(
UserID int NOT NULL,
AnimeID int NOT NULL,
PRIMARY KEY (AnimeID)
);

DROP TABLE IF EXISTS UserReadList;
CREATE TABLE UserReadList
(
UserID int NOT NULL,
MangaID int NOT NULL,
PRIMARY KEY (MangaID)
);
```

```sql
USE AnimeDatabase;

DROP TABLE IF EXISTS Users;
CREATE TABLE Users
(
FirstName varchar(20),
LastName varchar(20),
UserID int,
FriendID int,
AnimeReviewerID int,
MangaReviewerID int,
PRIMARY KEY (UserID)
);

DROP TABLE IF EXISTS Friends;
CREATE TABLE Friends
(
FirstName varchar(20),
LastName varchar(20),
FriendID int,
UserID int,
AnimeReviewerID int,
MangaReviewerID int,
PRIMARY KEY (UserID)
);

Insert INTO Users (FirstName, LastName, UserID, FriendID, AnimeReviewerID, MangaReviewerID) VALUES
('Ben', 'DeZutti', 1, 1, 1, 1),
('Cole', 'Mac', 2, 2, 2, 2),
('Alyssa', 'Cilvey', 3, 3, 3, 3),
('Nick', 'Wano', 4, 4, 4, 4),
('Greg', 'West', 5, 5, 5, 5);
```

Sorting of Anime and Manga (Select Statements)

```sql
SELECT * FROM Anime
ORDER BY Title ASC;

SELECT * FROM Anime
ORDER BY Genre ASC;

SELECT * FROM Anime
ORDER BY Studio ASC;

SELECT * FROM Anime
ORDER BY ReleaseDate DESC;

SELECT * FROM Anime
ORDER BY ReleaseDate ASC;

SELECT * FROM Anime
WHERE AnimeType = 'Movie';

SELECT * FROM Anime
WHERE AnimeType = 'Show';
```

```sql
SELECT * FROM Anime
WHERE AnimeType = 'Show';

SELECT * FROM Manga
ORDER BY Title ASC;

SELECT * FROM Manga
ORDER BY Genre ASC;

SELECT * FROM Manga
ORDER BY Publisher ASC;

SELECT * FROM Manga
ORDER BY ReleaseDate DESC;

SELECT * FROM Manga
ORDER BY ReleaseDate ASC;

SELECT * FROM Manga
WHERE BookType = 'Light Novel';
```

- Creates ability to categorize and sort anime/manga through several different means, allowing you to chose genre, alphabetical titles, what studio, and the recency of the data.

```sql
SELECT
    a.Title,
    m.Author,
    a.Studio
FROM Anime a
JOIN Manga m ON a.Title = m.Title;
```

- Join above merges anime and manga that have the same title showing the author of the manga and the studio that created it.

Adding New Anime/Manga & Deleting Anime/Manga

```
DELIMITER $$
CREATE PROCEDURE DeleteManga(
    IN p_MangaID INT
)
BEGIN
    -- Delete the manga with the specified MangaID from the Manga table
    DELETE FROM Manga
    WHERE MangaID = p_MangaID;
END$$

DELIMITER ;
```

```
DELIMITER $$
CREATE PROCEDURE AddNewAnime(
    IN p_AnimeID INT,
    IN p_Title VARCHAR(100),
    IN p_AnimeType VARCHAR(10),
    IN p_Seasons INT,
    IN p_Episodes INT,
    IN p_Genre VARCHAR(20),
    IN p_AnimeDescription VARCHAR(500),
    IN p_ReleaseDate DATE,
    IN p_Studio VARCHAR(30),
    IN p_Actors VARCHAR(100)
)

BEGIN
    -- Insert the new anime into the Anime table
    INSERT INTO Anime (AnimeID, Title, AnimeType, Seasons, Episodes, Genre, AnimeDescription, ReleaseDate, Studio, Actors)
    VALUES (p_AnimeID, p_Title, p_AnimeType, p_Seasons, p_Episodes, p_Genre, p_AnimeDescription, p_ReleaseDate, p_Studio, p_Actors);
END$$

DELIMITER ;
```

```
DELIMITER $$
CREATE PROCEDURE AddNewManga(
    IN p_MangaID INT,
    IN p_Title VARCHAR(100),
    IN p_BookType VARCHAR(20),
    IN p_Volumes INT,
    IN p_Chapters INT,
    IN p_Genre VARCHAR(20),
    IN p_BookDescription VARCHAR(500),
    IN p_ReleaseDate DATE,
    IN p_Publisher VARCHAR(30),
    IN p_Author VARCHAR(50)
)
BEGIN
    -- Insert the new manga into the Manga table
    INSERT INTO Manga (MangaID, Title, BookType, Volumes, Chapters, Genre, BookDescription, ReleaseDate, Publisher, Author)
    VALUES (p_MangaID, p_Title, p_BookType, p_Volumes, p_Chapters, p_Genre, p_BookDescription, p_ReleaseDate, p_Publisher, p_Author);
END$$
DELIMITER ;
```

Update Review Score (Update function in order to get review average)

```
CREATE PROCEDURE DeleteAnimeReview(
    In anime_id Int,
    In anime_reviewerid INT
)

BEGIN
    DELETE FROM AnimeReviews
    WHERE AnimeMatchID = anime_Id AND AnimeReviewerID = anime_reviewerid ;
END

DROP TRIGGER IF EXISTS updateSummary;
CREATE TRIGGER updateSummary
AFTER DELETE ON AnimeReview
BEGIN
    UPDATE AnimeReviewSummary
    SET TotalReviews = TotalReviews - 1,
        AverageReview = IF(TotalReviews > 0, (SELECT SUM(AnimeStarRating) FROM AnimeReviews WHERE AnimeID = OLD.AnimeID) / Total_Reviews, 0)
    WHERE AnimeID = OLD.AnimeID;
END
```

```
DROP PROCEDURE IF EXISTS AddNewAnimeReview;
CREATE PROCEDURE AddNewAnimeReview(
    IN p_AnimeID INT,
    IN p_AnimeReviewerID INT,
    IN p_AnimeReviewBody VARCHAR(100),
    IN p_AnimeStarRating INT,
    IN p_AnimeReviewLikeCount INT
)
BEGIN
    -- Insert the new review into AnimeReviews table
    INSERT INTO AnimeReviews (AnimeMatchID, AnimeReviewerID, AnimeReviewBody, AnimeStarRating, AnimeReviewLikeCount)
    VALUES (p_AnimeID, p_AnimeReviewerID, p_AnimeReviewBody, p_AnimeStarRating, p_AnimeReviewLikeCount);

END $$

DROP TRIGGER IF EXISTS updateSummary;
CREATE TRIGGER updateSummary
AFTER INSERT ON AnimeReviews
FOR EACH ROW
BEGIN
    -- Declare variables to hold total reviews and sum of ratings
    DECLARE total_reviews INT;
    DECLARE total_rating INT;

    -- Get the current total reviews and total ratings for the given AnimeID
    SELECT COUNT(*), SUM(AnimeStarRating)
    INTO total_reviews, total_rating
    FROM AnimeReviews
    WHERE AnimeMatchID = p_AnimeID;

    -- If there's no entry for this AnimeID, initialize the values
    IF total_reviews IS NULL THEN
        SET total_reviews = 0;
        SET total_rating = 0;
    END IF;

    -- Calculate the average rating
    DECLARE average_rating DECIMAL(3, 2);
    SET average_rating = IF(total_reviews > 0, total_rating / total_reviews, 0);

    -- Update or insert the summary into the AnimeReviewSummary table
    INSERT INTO AnimeReviewSummary (AnimeID, TotalReviews, AverageReview)
    VALUES (NEW.AnimeMatchID, total_reviews, average_rating)
    ON DUPLICATE KEY UPDATE
        TotalReviews = total_reviews,
        AverageReview = average_rating;
END
```

Adding Friends, Deleting/Adding User accounts, Add to Watch List (Procedures)

```
DROP PROCEDURE IF EXISTS AddToWatchList;
DELIMITER $$
CREATE PROCEDURE AddToWatchList(
    In anime_id Int,
  user_id Int
)
```

```
DROP PROCEDURE IF EXISTS DeleteFromWatchList;
CREATE PROCEDURE DeleteFromWatchList(
    In anime_id Int,
  user_id Int
)
```

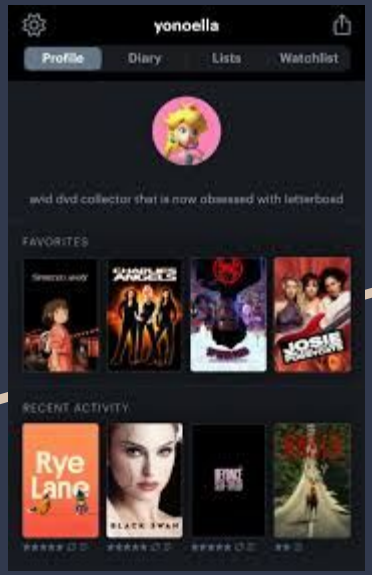
```
#Create user account
DROP PROCEDURE IF EXISTS CreateFriend;
DELIMITER $$
CREATE PROCEDURE CreateFriend (IN fName VARCHAR(20), IN lName VARCHAR(20), IN userID INT, IN friendID INT, IN animeReviewerID INT, IN mangaReviewerID INT)
BEGIN
    INSERT INTO Friends (FirstName, LastName, UserID, FriendID, AnimeReviewerID, MangaReviewerID)
    VALUES (fName, lName, userID, friendID, animeReviewerID, mangaReviewerID);
END $$
DELIMITER ;
```

```
USE AnimeDatabase;

#Create user account
DROP PROCEDURE IF EXISTS CreateNewUser;
DELIMITER $$
CREATE PROCEDURE CreateNewUser (IN fName VARCHAR(20), IN lName VARCHAR(20), IN userID INT, IN friendID INT, IN animeReviewerID INT, IN mangaReviewerID INT)
BEGIN
    INSERT INTO Users (FirstName, LastName, UserID, FriendID, AnimeReviewerID, MangaReviewerID)
    VALUES (fName, lName, userID, friendID, animeReviewerID, mangaReviewerID);
END $$
DELIMITER ;

#Delete user account
DROP PROCEDURE IF EXISTS DeleteUser;
DELIMITER $$
CREATE PROCEDURE DeleteUser (IN id int)
BEGIN
    DELETE FROM Users WHERE Users.UserID = id;
END $$
DELIMITER ;
```

What else would we like to add in the future?



- I'd like to add an option for account personality (Display a user's favorite Anime, favorite manga, favorite characters, favorite genre, etc.)
- Create an ability to like and comment on other people's reviews
- Create an ability for studios/authors to alert fans of their work that they are coming out with something new.
- Get Friends Working
- Fix bugs in the average score
- Better Organization of Procedures and Database